AI Audit Report

Declaration

I use AI tools for requirements analysis, candidate test generation, correction, and automation/report scaffolding. The browser session used Google Gemini Pro, signed in as Hưng Nguyễn; the email address is deliberately redacted. Local automation assistance used OpenAI Codex and is disclosed in the main report. All Gemini interactions below include tool, time, prompt, and complete output.

Session index

Password reset

Interaction	Sent UTC	Sent Asia/Ho_Chi_Minh	Response completed UTC
RESET-P1	2026-08-20T17:51:38.433Z	2026-08-21, 00:51:38	2026-08-20T17:52:27.923Z
RESET-P2	2026-08-20T17:52:42.872Z	2026-08-21, 00:52:42	2026-08-20T17:53:36.433Z
RESET-P3	2026-08-20T17:53:52.570Z	2026-08-21, 00:53:52	2026-08-20T17:55:04.844Z

- Verbatim prompt + output: reset-password-session.txt
- Timestamp metadata: reset-password-timestamps.json

Apply coupon

Interaction	Sent UTC	Sent Asia/Ho_Chi_Minh	Response completed UTC
COUPON-P1	2026-08-20T17:55:59.484Z	2026-08-21, 00:55:59	2026-08-20T17:57:09.784Z
COUPON-P2	2026-08-20T17:57:31.946Z	2026-08-21, 00:57:31	2026-08-20T17:58:37.609Z
COUPON-P3	2026-08-20T17:58:51.861Z	2026-08-21, 00:58:51	2026-08-20T18:00:15.992Z
COUPON-P4	2026-08-20T18:00:35.477Z	2026-08-21, 01:00:35	2026-08-20T18:01:15.694Z

- Verbatim prompt + output: apply-coupon-session.txt
- Timestamp metadata: apply-coupon-timestamps.json

Import products

Interaction	Sent UTC	Sent Asia/Ho_Chi_Minh	Response completed UTC
IMPORT-P1	2026-08-20T18:01:51.998Z	2026-08-21, 01:01:51	2026-08-20T18:03:04.779Z
IMPORT-P2	2026-08-20T18:03:24.673Z	2026-08-21, 01:03:24	2026-08-20T18:04:09.712Z
IMPORT-P3	2026-08-20T18:04:25.153Z	2026-08-21, 01:04:25	2026-08-20T18:05:35.085Z
IMPORT-P4	2026-08-20T18:05:57.495Z	2026-08-21, 01:05:57	2026-08-20T18:06:37.703Z

- Verbatim prompt + output: import-products-session.txt
- Timestamp metadata: import-products-timestamps.json

Review trail

- Reset follow-up corrected invented status codes, arbitrary maximum-password assumptions, RFC 5322 overreach, and the OTP state model.

- Coupon follow-ups separated exact schema from observations, rejected floating values where the SUT contract uses currency integers, made authentication gating explicit, and corrected the false “apply consumes usage” assumption.
- Import follow-ups separated frontend CSV parsing from the JSON API, clarified response-count and atomicity semantics, kept SQL literals valid, and added eight supplemental candidates for missed partitions.
- Final executable audit labels and reasons are stored per row in the Excel workbook and `scripts/case-definitions.mjs`.

Screenshots: `evidence/screenshots/01-gemini-reset-candidates.png`, `02-gemini-coupon-correction.png`, and `03-gemini-import-correction.png`.

Verbatim transcript — Password reset

```
Trò chuy■n v■i Gemini
B■n ■ã nói
You are assisting with an individual university API-testing assignment. Work only from the authoritative requirements below. This is Phase 1: requirements analysis for exactly one target API. Do NOT generate test cases yet.
TARGET
POST /api/reset-password
JSON body:
{
"email": "test@domain.com",
"resetToken": "123456",
"newPassword": "NewPassword123!"
}
BUSINESS REQUIREMENTS
FR-03 is a two-step password reset.
Step 1: a registered user requests an OTP for an email address. The system generates a random 6-digit OTP.
Step 2: the user provides OTP, new password, and confirmation of new password.
The new password follows FR-01: minimum 8 characters and at least one uppercase letter, one lowercase letter, one digit, and one special character from @ $ ! % * ? &.
The two passwords must match.
The OTP is bound to the email that requested it and cannot be used for another email.
SECURITY REQUIREMENTS
SEC-01 passwords must not be stored as plaintext.
SEC-05 database queries must be parameterized.
SEC-07 reset OTP must have at least 6 digits of entropy, must expire, and must be invalidated after successful use.
API SPECIFICATION
The API request has only email, resetToken, and newPassword. The API specification does not expose a confirm-password field and does not define exact error status codes or an error schema.
ASSIGNMENT EXPECTATIONS
Later work must cover domain partitions for every parameter, workflow/state transitions, SEC-01 through SEC-07 where applicable, and exact response-schema validation. Ambiguities must be identified rather than invented.
Produce:
1. A requirement-traceability table with requirement ID, observable API rule, precondition, and ambiguity.
2. A parameter/domain model for email, resetToken, and newPassword.
3. A state model for OTP lifecycle.
4. A list of explicit assumptions that must be confirmed or handled conservatively.
Do not inspect implementation behavior and do not generate test cases in this phase.
Gemini ■ã nói
Here is the Phase 1 Requirements Analysis for the /api/reset-password endpoint. As requested, this focuses entirely on requirement decomposition, domain modeling, and state transitions without generating executable test cases.
1. Requirement-Traceability Table
Requirement ID Observable API Rule Precondition Ambiguity / Conflict
FR-03 (A) The API accepts an OTP (resetToken) and a newPassword to complete the reset.
A valid OTP must have been requested for the user in Step 1. Conflict: FR-03 states the user provides a "confirmation of new password," but the API spec lacks this field. It is unclear if the API intentionally delegates this to the client-side or if the spec is incomplete.
FR-03 (B) The OTP must be bound to the specific email that requested it. The OTP exists and is in an Active state.
None, though exact error codes for an email/OTP mismatch are undefined.
FR-01
newPassword must be ≥ 8 chars, containing ≥ 1 uppercase, 1 lowercase, 1 digit, and 1 specific special character (@ $ ! % * ? &).
```

None. Ambiguity: What is the maximum password length? Are special characters outside the allowed list rejected, or is the presence of at least one from the list sufficient?

SEC-01 Passwords must not be stored in plaintext. Password successfully reset. Ambiguity: Cannot be strictly validated via black-box API testing unless a side-channel or secondary API leaks the hash/plaintext.

SEC-05 Database queries must be parameterized to prevent SQL injection. API accepts input payloads.
None, though black-box testing can only infer compliance by observing behavior against malicious payloads.

SEC-07 (A) OTP must have at least 6 digits of entropy. OTP generated in Step 1.
Conflict: FR-03 dictates a "6-digit OTP", but a 6-digit numeric OTP (10

6

 combinations) has ≈19.9 bits of entropy, which is standard but might conflict with stricter interpretations of "6 digits of entropy."

SEC-07 (B) OTP must expire and must be invalidated after successful use. OTP is in an Active state.
Ambiguity: The exact Time-To-Live (TTL) for expiration is not defined in the business requirements.

2. Parameter & Domain Model

This model defines the partitions for the three input parameters based on the combined requirements.

Parameter Valid Classes (In-Domain) Invalid Classes (Out-of-Domain)

email

- Registered email with active OTP

- Standard RFC 5322 format

- Unregistered email

- Registered email with no active OTP

- Malformed email string

- Null / Missing / Empty

resetToken

- 6-digit numeric string (e.g., "123456")

- Matches the provided email

- Unexpired & Unused

- Length <6 or >6

- Contains non-numeric characters

- Matches a different user's email

- Expired or previously used

- Null / Missing / Empty

newPassword

- Length: 8≤L≤Max

- Contains at least one [A-Z]

- Contains at least one [a-z]

- Contains at least one [0-9]

- Contains at least one of @ $! % * ? &

- Length <8

- Missing uppercase

- Missing lowercase

- Missing digit

- Missing required special character

- Null / Missing / Empty

3. State Model (OTP Lifecycle)

The resetToken (OTP) operates as a state machine. Because the API binds the token to an email and enforces lifecycle rules (SEC-07), its state directly dictates the API's response behavior.

State 1: Non-Existent / Null

Condition: Before Step 1 is executed, or if an invalid token is provided.

Transition: Moves to Active when the Step 1 generation API is called.

State 2: Active

Condition: OTP generated, bound to email, time <TTL.

Transitions:

Moves to Invalidated (Used) upon a successful POST /api/reset-password request.

Moves to Expired if time ≥TTL.

Self-Transition: Remains Active if POST /api/reset-password fails due to client error (e.g., newPassword violates FR-01). (See Assumption 4)

State 3: Expired (Terminal)

Condition: The system clock exceeds the OTP's assigned TTL.

Transition: None. Token is permanently rejected.

State 4: Invalidated / Used (Terminal)

Condition: The token was successfully consumed by FR-03 Step 2.

Transition: None. Token is burned (SEC-07) and cannot be reused.

4. Explicit Assumptions

Because the requirements contain gaps and conflicts, the following assumptions must be documented for conservative test planning. If they cannot be confirmed by the Product Owner/Architect, test cases must handle them dynamically:

Missing Confirmation Field: Because the API spec lacks a confirmPassword field, we assume this validation is delegated entirely to the frontend client, and the API is only responsible for updating the single newPassword provided.

Undefined Error Specifications: Because standard HTTP status codes and error schemas are omitted, we assume standard RESTful semantics for testing purposes (e.g., 400 Bad Request for FR-01 complexity violations, 401/403 for invalid/expired tokens, 404 for unknown emails).

OTP Burn Behavior on Failure: We assume an OTP remains in the Active state if a request fails due to payload validation (e.g., an invalid password format). If the system aggressively burns the OTP on any attempt (failed or successful), the testing workflow will require generating a new OTP (Step 1) before every negative test case.

Maximum Password Length: We assume there is an undocumented maximum string length for newPassword (e.g., 64 or 72 characters) to prevent Denial of Service (DoS) attacks against the backend hashing algorithm (bcrypt/Argon2 limitations).

Strict Data Types: We assume the API strictly expects a JSON payload with Content-Type: application/json. Testing will need to verify how it handles type mismatches (e.g., sending resetToken as an integer 123456 instead of a string "123456").

B■n ■ã nói

Review your Phase 1 response critically and revise it. Two problems must be corrected:

1. You assumed exact HTTP codes (400, 401/403, 404), but the supplied specification does not define them. These cannot be pass/fail oracles. Separate normative assertions from observational fields that require human review.

2. You invented a maximum password length (64/72), which is not stated. A large-password check may be a robustness/security exploration, but it cannot be classified as a requirements-based invalid equivalence class.

3. You used RFC 5322 as though it were specified. The only supplied format example is user@domain.com, so identify broader email-format variants as ambiguity/robustness partitions.

4. Clarify that SEC-01 cannot be proven solely through this endpoint response; it needs authorized white-box/database evidence and must not be inferred merely because login succeeds.

5. “6 digits of entropy” in SEC-07 should be operationalized conservatively as a six-digit numeric OTP because FR-03 explicitly says so; keep the entropy-language concern as a note, not as a conflicting test oracle.

Produce a corrected version with:

A. Normative black-box oracles.

B. Observational/robustness checks requiring human review.

C. A decision table combining email binding, OTP existence/format/expiry/use state, and password complexity.

D. A minimal response-schema oracle that does not invent undocumented fields or status codes.

Still do not generate individual test cases.

Gemini ■ã nói

Here is the revised Phase 1 Requirements Analysis, correcting the previous assumptions and separating strict normative requirements from exploratory and observational concerns.

A. Normative Black-Box Oracles

These are the strict pass/fail criteria that can be evaluated dynamically via the API based only on the provided specification.

Positive Outcome (State Change): When email is registered, resetToken is a 6-digit numeric string that is Active, unexpired, and bound to that specific email, and newPassword meets all FR-01 criteria (minimum 8 characters, ≥ 1 uppercase, ≥ 1 lowercase, ≥ 1 digit, and ≥ 1 allowed special character), the request MUST be accepted. The system state MUST change to allow subsequent authentication with the new password.

Token Invalidation (SEC-07): Following a successful positive outcome, a subsequent request using the identical resetToken MUST be rejected.

Negative Outcome - Password Complexity (FR-01): If newPassword lacks any required character class or is under 8 characters, the request MUST be rejected (the password state remains unchanged).

Negative Outcome - Token Binding (FR-03): If the resetToken is Active but bound to a different email address than the one provided in the payload, the request MUST be rejected.

Negative Outcome - Token Lifecycle (SEC-07): If the resetToken is expired, the request MUST be rejected.

Negative Outcome - Token Format: If the resetToken is not exactly 6 digits (operationalized strictly as a numeric string per FR-03, addressing the "6 digits of entropy" requirement conservatively), it MUST be rejected.

B. Observational & Robustness Checks (Requires Human Review)

These checks explore ambiguities, undefined behaviors, and security requirements that cannot be strictly failed by a black-box API test without further specification or white-box access.

HTTP Status Codes & Error Schemas: Because the specification does not define specific HTTP status codes (e.g., 400, 401, 404) or an error response schema, the testing tool will capture the returned status codes, headers, and payload structures for manual review. We cannot automatically fail the API for returning a 200 OK with an error message in the body, nor for returning a 403 vs a

401.

Maximum Password Length (Robustness): The specification does not define a maximum password length. Tests will submit excessively large strings (e.g., 1,000+ characters) to observe for crashes, timeouts, or unhandled exceptions (DoS vulnerabilities), but these inputs do not form a strict "invalid" equivalence class based on FR-01.

Email Format Variations (Ambiguity): The only provided specification for email is the string format "test@domain.com". Tests will probe broader formats (e.g., subdomains, plus-addressing, special characters) to observe how the API interprets email validity, but we cannot establish strict pass/fail boundaries for RFC 5322 compliance without clarification.

Confirm Password Field (FR-03 Conflict): The API spec omits the confirmation field mentioned in FR-03. We will observe and document that the API functions without it, operating under the assumption that this check is delegated to the client layer.

Data-at-Rest Security (SEC-01): Validating that passwords are not stored in plaintext cannot be proven by observing a successful API response. This requires out-of-band verification (e.g., DBA confirmation, white-box database querying, or reviewing source code).

SQL Injection (SEC-05): We will inject standard SQLi payloads into all parameters as a robustness check. However, definitive proof of parameterized queries requires code-level or database-log review.

C. Decision Table: Logic & State Combinations

This table models the core business logic intersections. "Accept" and "Reject" are defined by the normative oracles in Section A.

Rule Email Exists resetToken State & Binding newPassword Meets FR-01 Expected Action

1 Yes Valid (6-digit, Active, Matches Email) Yes Accept (Reset password, burn token)

2 Yes Valid (6-digit, Active, Matches Email) No Reject

3 Yes Valid, but bound to different email Yes Reject

4 Yes Expired Yes Reject

5 Yes Already Used (Invalidated) Yes Reject

6 Yes Invalid Format (e.g., 5-digit, alpha) Yes Reject

7 No Any Yes Reject

D. Minimal Response-Schema Oracle

Because no success or error schemas are documented, the automated response oracle must be strictly limited to universally applicable API behavior to avoid inventing requirements:

Valid JSON: If a response body is returned, it must be syntactically valid JSON.

No Unhandled Exceptions: The response must not leak stack traces, database errors, or server framework debug information.

State Verification (The True Oracle): For any positive test (Rule 1), the true oracle is a subsequent API call to the system's authentication endpoint using the newPassword. If authentication succeeds, the reset was successful. For negative tests, a subsequent authentication attempt with the newPassword must fail, proving the state was not altered.

B■n ■ã nói

Now generate the AI candidate test suite for POST /api/reset-password from the corrected analysis. Produce exactly 42 distinct cases so the final audited suite can meet the assignment target of at least 35 cases.

Use a compact Markdown table with these columns:

AI_ID, Title, Preconditions/OTP state, Request variation, Expected business outcome (ACCEPT/REJECT/OBSERVE), Automated oracle, Requirement IDs, Technique, Priority.

Rules:

- Cover every parameter independently: missing, null, empty, wrong JSON type, and meaningful boundary/equivalence classes.

- Cover password length 7 and 8, each missing character class, each allowed special character category as appropriate, Unicode and whitespace observations, and a large input robustness check.

- Cover OTP six-digit format boundaries, leading zero, wrong email binding, nonexistent, expired, used/replay, newly reissued OTP replacing an older OTP, and concurrent/repeated requests.

- Include SQL injection and JSON/property manipulation checks, but do not claim they prove SEC-05.

- Include positive state verification via login and negative state-preservation verification.

- Do not invent exact HTTP status codes or undocumented response fields. Use “non-success outcome + state unchanged” where normative, and record actual status/body for observation.

- Mark tests that require time control, database inspection, or concurrency in the Automated oracle field.

- Do not label any cases as student-authored; all 42 in this response are AI-generated candidates.

After the table, provide a coverage count by parameter, state, security, and schema/robustness.

Gemini ■ã nói

Here is the candidate test suite containing exactly 42 distinct test cases for the POST /api/reset-password endpoint.

AI Candidate Test Suite

AI_ID Title Preconditions/OTP state Request variation Expected business outcome Automated oracle Requirement IDs Technique Priority

TC-01 Base Positive Reset Registered email, OTP Active 8 chars, 1 upper, 1 lower, 1 digit, 1 allowed special (@) ACCEPT Auth check confirms login with new password FR-03, FR-01 Equivalence Partitioning High

TC-02 OTP Leading Zero Registered email, OTP Active starts with 0 resetToken = 012345 ACCEPT Auth check confirms login FR-03 Boundary Value High

TC-03 All Allowed Special Chars Registered email, OTP Active newPassword contains @$!%*?& ACCEPT Auth check confirms login FR-01 Equivalence Partitioning Medium

TC-04 Valid Reissued OTP Email has 2 OTPs, only newest is Active resetToken uses newest OTP ACCEPT Auth check confirms login FR-03, SEC-07 State Transition High

TC-05 Password Length 7 Registered email, OTP Active newPassword length = 7 (all classes met) REJECT Non-success outcome + auth check confirms no change FR-01 Boundary Value High

TC-06 Missing Uppercase Registered email, OTP Active newPassword lacks [A-Z] REJECT Non-success outcome + auth check confirms no change FR-01 Equivalence Partitioning High

TC-07 Missing Lowercase Registered email, OTP Active newPassword lacks [a-z] REJECT
Non-success outcome + auth check confirms no change FR-01 Equivalence Partitioning High

TC-08 Missing Digit Registered email, OTP Active newPassword lacks [0-9] REJECT
Non-success outcome + auth check confirms no change FR-01 Equivalence Partitioning High

TC-09 Missing Special Char Registered email, OTP Active newPassword lacks special char REJECT
Non-success outcome + auth check confirms no change FR-01 Equivalence Partitioning High

TC-10 Unlisted Special Char Registered email, OTP Active newPassword uses # instead of allowed chars REJECT
Non-success outcome + auth check confirms no change FR-01 Equivalence Partitioning Medium

TC-11 Leading/Trailing Space Registered email, OTP Active newPassword padded with spaces OBSERVE
Check if accepted (and if spaces are stripped or hashed) via Auth FR-01 Robustness Medium

TC-12 Internal Whitespace Registered email, OTP Active newPassword contains internal space OBSERVE
Check if accepted via Auth login FR-01 Robustness Medium

TC-13 Unicode/Emoji Registered email, OTP Active newPassword includes emoji OBSERVE Check if accepted via Auth login FR-01
Robustness Low

TC-14 Large Password Registered email, OTP Active newPassword > 1000 chars OBSERVE Capture response status/body; verify no crash
FR-01 Robustness Low

TC-15 Expired OTP Registered email, OTP Expired Valid payload, OTP past TTL REJECT
[Time control required] Non-success + no state change SEC-07 State Transition High

TC-16 Replay/Used OTP Registered email, OTP Invalidated Resubmit payload from TC-01 REJECT
Non-success + auth check confirms no change SEC-07 State Transition High

TC-17 Older Replaced OTP Email has 2 OTPs, older is Invalidated resetToken uses the older OTP REJECT
Non-success + auth check confirms no change SEC-07 State Transition High

TC-18 Wrong Email Binding 2 Registered emails, both Active OTPs Swap valid email with the other's resetToken REJECT
Non-success + auth check confirms no change FR-03 Decision Table High

TC-19 Nonexistent OTP Registered email, No Active OTP resetToken = random 6 digits not in DB REJECT
Non-success + auth check confirms no change FR-03 Decision Table High

TC-20 OTP Length 5 Registered email, OTP Active resetToken length = 5 (truncated) REJECT
Non-success + auth check confirms no change FR-03, SEC-07 Boundary Value High

TC-21 OTP Length 7 Registered email, OTP Active resetToken length = 7 (appended) REJECT
Non-success + auth check confirms no change FR-03, SEC-07 Boundary Value High

TC-22 Alphanumeric OTP Registered email, OTP Active resetToken contains letters REJECT
Non-success + auth check confirms no change FR-03, SEC-07 Equivalence Partitioning High

TC-23 Concurrent OTP Usage Registered email, OTP Active Send same payload in parallel OBSERVE
[Concurrency required] Exactly 1 accepts, others reject SEC-07 State Transition Medium

TC-24 Unregistered Email Unregistered email Valid format for unregistered user REJECT
Non-success (observe if 404 vs generic error for enum) FR-03 Decision Table High

TC-25 Subdomain Email Registered email with subdomain, OTP Active e.g., test@sub.domain.com OBSERVE
Check if accepted via Auth login FR-03 Robustness Low

TC-26 Case-Sensitive Email Registered email, OTP Active email value capitalized OBSERVE Check if accepted via Auth login FR-03
Robustness Medium

TC-27 Missing Email Key Registered email, OTP Active Remove email property REJECT Non-success + valid JSON response Schema
Negative Medium

TC-28 Null Email Registered email, OTP Active email = null REJECT Non-success + valid JSON response Schema Negative Medium

TC-29 Empty Email Registered email, OTP Active email = "" REJECT Non-success + valid JSON response Schema Negative Medium

TC-30 Wrong Type Email Registered email, OTP Active email = integer/boolean REJECT Non-success + valid JSON response Schema
Negative Low

TC-31 Missing Token Key Registered email, OTP Active Remove resetToken property REJECT Non-success + valid JSON response Schema
Negative Medium

TC-32 Null Token Registered email, OTP Active resetToken = null REJECT Non-success + valid JSON response Schema Negative Medium

TC-33 Empty Token Registered email, OTP Active resetToken = "" REJECT Non-success + valid JSON response Schema Negative Medium

TC-34 Wrong Type Token Registered email, OTP Active resetToken = integer 123456 OBSERVE Capture if strictly typed or auto-cast
Schema Negative Low

TC-35 Missing Password Key Registered email, OTP Active Remove newPassword property REJECT Non-success + valid JSON response
Schema Negative Medium

TC-36 Null Password Registered email, OTP Active newPassword = null REJECT Non-success + valid JSON response Schema Negative
Medium

TC-37 Empty Password Registered email, OTP Active newPassword = "" REJECT Non-success + valid JSON response Schema Negative Medium

TC-38 Wrong Type Password Registered email, OTP Active newPassword = integer REJECT Non-success + valid JSON response Schema
Negative Low

TC-39 Undocumented Prop Registered email, OTP Active Inject "isAdmin": true OBSERVE Observe if ignored or rejected Schema
Robustness Low

TC-40 Malformed JSON N/A Missing closing brace REJECT Non-success + observe status/error leak Schema Negative High

TC-41 SQLi in Email N/A email = ' OR 1=1 -- REJECT Non-success + [DB inspection to verify SEC-05] SEC-05 Security High

TC-42 SQLi in Token Registered email, OTP Active resetToken = 123456' REJECT Non-success + [DB inspection to verify SEC-05] SEC-05
Security High

Coverage Summary

Parameter Coverage (14 cases):

Valid, invalid, missing, null, empty, and wrong type variations for email (4), resetToken (5), and newPassword (5).

State & Lifecycle Coverage (5 cases):

Active/Valid, Expired, Used/Replayed, Superseded (reissued), and Concurrent usage.

Security & Constraint Coverage (15 cases):

FR-01 Password complexity limits & omissions (6).

FR-03/SEC-07 OTP format boundaries and email binding violations (7).

SEC-05 Injection payload behaviors requiring out-of-band verification (2).

Schema & Robustness Coverage (8 cases):

Encoding/Whitespace, unicode limits, oversized payloads, JSON parsing errors, and undocumented property injection.

Pro

Gemini là AI và có th■ m■c sai sót.

Verbatim transcript — Apply coupon

Trò chuv■n v■i Gemini

B■n ■ã nói

You are assisting with an individual university API-testing assignment. Work only from the authoritative requirements below. This is Phase 1: requirements analysis for exactly one target API. Do NOT generate test cases yet.

TARGET

POST /api/apply-coupon

JSON body:

{

"code": "SAVE10",

"total_amount": 500000,

"user_id": 1

}

Successful response is documented only as JSON containing discount_amount and final_amount.

FR-09 BUSINESS RULES

A coupon applies only when all five conditions are true:

C1 code exists and is_active = 1.

C2 current date is before expired_at.

C3 total_amount >= min_order_amount.

C4 user is authenticated with a valid JWT.

C5 this user's usage count for this coupon is < max_uses_per_user.

FORMULAS

percent: discount_amount = total_amount * discount_value / 100.

fixed: discount_amount = discount_value.

final_amount = total_amount - discount_amount.

SEEDED COUPONS

SAVE10: percent 10, minimum 300000, expires 2099-12-31, maximum uses 1.

BIGBUY: fixed 50000, minimum 500000, expires 2099-12-31, maximum uses 1.

VIP100: fixed 100000, minimum 300000, expires 2099-12-31, maximum uses 2.

EXPIRED: percent 20, minimum 100000, expires 2020-01-01, maximum uses 1.

SECURITY

SEC-02 protected APIs require a valid JWT.

SEC-05 database queries must be parameterized.

The body includes user_id, but FR-09 says authentication is by JWT. Treat the authenticated JWT identity as authoritative and fla g the body user_id as an IDOR/trust-boundary ambiguity.

The specification does not define exact error status codes or the error schema.

ASSIGNMENT EXPECTATIONS

Later work must cover every parameter domain, decision combinations, security, exact documented success schema, and robustness. A mbiguities must not be converted into invented requirements.

Produce:

1. Requirement traceability.

2. Equivalence/boundary partitions for code, total_amount, user_id, and Authorization.

3. A decision table that isolates each C1-C5 condition plus important interactions.

4. Formula oracles including boundary total == minimum.

5. Explicit ambiguities and what should be normative versus observational.

Do not inspect implementation behavior and do not generate individual test cases yet.

Gemini ■ã nói

Here is the Phase 1 Requirements Analysis for the POST /api/apply-coupon API. This analysis strictly bounds the scope to the prov ided specifications without generating explicit test cases.

1. Requirement Traceability Matrix

ID Source Description

REQ-01 Target Endpoint is POST /api/apply-coupon accepting JSON (code, total_amount, user_id).

REQ-02 Target Successful response must contain exactly discount_amount and final_amount.

REQ-03 FR-09 (C1) Coupon must exist and have is_active = 1.

REQ-04 FR-09 (C2) Current date must be strictly before expired_at.

REQ-05 FR-09 (C3) Request total_amount must be >= min_order_amount.

REQ-06 FR-09 (C4) Request requires a valid JWT for user authentication.

REQ-07 FR-09 (C5) User's usage_count for the requested coupon must be < max_uses_per_user.

REQ-08 Formulas discount_amount is calculated by percentage or fixed value; final_amount = total_amount - discount_amount.

REQ-09 SEC-02 The API is protected and requires a valid JWT.

REQ-10 SEC-05 Database queries must be parameterized to prevent SQL Injection.

REQ-11 Security IDOR conflict: Authenticated JWT identity is authoritative over the body user_id.

2. Equivalence & Boundary Partitions

code (String)

Valid: Existing, active, unexpired coupons (e.g., SAVE10, BIGBUY, VIP100).

Invalid (Business Logic): Expired coupons (EXPIRED), non-existent codes (FAKE50).

Invalid (Robustness/Security): Empty string "", null, extremely long strings, strings with SQL injection payloads (to test REQ-10).

total_amount (Numeric)

Valid (Nominal): Values strictly greater than min_order_amount (e.g., 500000 for SAVE10).

Valid (Boundary): Values exactly equal to min_order_amount (e.g., 300000 for SAVE10).

Invalid (Boundary): Values exactly one unit below min_order_amount (e.g., 299999 for SAVE10).

Invalid (Domain/Robustness): Negative numbers, 0, missing key, null, non-numeric strings, float values (if integer is strictly expected).

user_id (Numeric)

Valid: Matches the user ID extracted from the provided JWT.

Invalid (Security/IDOR): Does not match the user ID extracted from the JWT.

Invalid (Robustness): Missing key, null, non-numeric values, negative IDs.

Authorization Header (JWT)

Valid: Well-formed Bearer <token> with valid signature and unexpired claims.

Invalid: Missing header, expired JWT, invalid signature, malformed token structure.

3. Decision Table (FR-09 Conditions)

This table isolates C1 through C5 to ensure every business rule boundary is handled.

(T = True, F = False, - = Don't Care)

Rule C1 (Active/Exists) C2 (Not Expired) C3 (Amount >= Min) C4 (Valid JWT) C5 (Under Max Uses) Expected Outcome

1 (Happy Path) T T T T T Success: Calculate & return amounts

2 (Invalid Code) F - - - - Error: Coupon invalid or not found

3 (Expired) T F - - - Error: Coupon expired

4 (Amount Too Low) T T F - - Error: Minimum order amount not met

5 (Unauthorized) T T T F - Error: Authentication failed

6 (Max Uses Hit) T T T T F Error: Usage limit exceeded

4. Formula Oracles

These oracles define expected mathematical outcomes, specifically hitting the total == minimum boundaries required by the business rules.

Coupon Type Total Amount Input Discount Calculation Expected discount_amount Expected final_amount

SAVE10 Percent 300000 (Boundary) 300000 * 10 / 100 30000 270000

SAVE10 Percent 500000 (Nominal) 500000 * 10 / 100 50000 450000

BIGBUY Fixed 500000 (Boundary) Fixed 50000 50000 450000

VIP100 Fixed 300000 (Boundary) Fixed 100000 100000 200000

5. Explicit Ambiguities & Testing Strategy

To proceed to Phase 2 (Test Generation), the following undocumented behaviors must be categorized as either Normative (strictly asserted based on standards) or Observational (discovered through exploratory testing and logged as actual behavior).

Error Status Codes & Schema (Observational)

Ambiguity: The specification only defines the success JSON schema. It does not state whether errors return 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, or 422 Unprocessable Entity.

Strategy: Tests will assert that the API fails gracefully (does not apply the coupon) but will only observe the HTTP status code and error schema for reporting purposes.

The user_id Trust Boundary (Normative)

Ambiguity: The payload requires user_id, but FR-09 mandates JWT authentication.

Strategy: A request where JWT_Identity != Body_user_id represents a direct IDOR vulnerability attempt. Tests will treat the JWT as authoritative. The expected normative behavior is that the API strictly rejects the request (or forcefully overrides the payload with the JWT identity) rather than processing the transaction for a different user.

Floating-Point Math / Rounding (Observational)

Ambiguity: If total_amount is 300005 with a 10% discount, discount_amount is mathematically 30000.5.

Strategy: The specification does not define integer rounding rules (e.g., floor, ceiling, or nearest). Tests using non-roundable numbers will record the rounding implementation chosen by the developer.

Order of Evaluation / Precedence (Observational)

Ambiguity: If a request features an expired token (C4 fails) AND an invalid code (C1 fails), which error does the API return first?

Strategy: Observe and document the API's validation short-circuiting sequence.

B■n ■ã nói

Critically revise your coupon analysis before test generation.

Correct these issues:

1. The success schema says the JSON contains discount_amount and final_amount; it does not say those are the only fields. The normative oracle is “both required numeric fields are present and correct,” while additional fields are observational unless they leak sensitive data.

2. No integer-only rule is supplied for total_amount. Floats and rounding are observational, not requirements-based invalid classes.

3. FR-09 C4 and SEC-02 mean authentication is mandatory for the API itself. In the decision table, do not use C4 as “don't care” when another condition fails; an unauthenticated caller must not be allowed to probe coupon existence, expiry, thresholds, or usage.

4. The body user_id conflicts with JWT identity. Separate two defensible behaviors for a mismatch: reject it or ignore it and use JWT identity. Either is safe; using the body identity for C5 is a failure.

5. Distinguish failure to apply the coupon from specific undocumented status/error payloads.

6. Add interactions for exact threshold, usage count max-1 versus max, percent/fixed formulas, expired JWT, user A JWT plus user B body ID, and missing user_id with valid JWT.

Return:

A. Revised normative oracles.

B. Revised decision table with authentication gating.

C. A trust-boundary table for JWT identity versus body user_id.

D. Formula/schema invariants.

Still do not generate individual test cases.

Gemini ■ã nói

Here is the critically revised Phase 1 Requirements Analysis, incorporating the corrected bounds for schema validation, data types, authentication gating, and trust-boundary conflicts.

A. Revised Normative Oracles

These oracles separate strict, required behaviors (Normative) from undocumented behaviors that will merely be recorded (Observational).

Success Oracle (Normative): When conditions C1–C5 are met, the API MUST apply the coupon and return a JSON response containing at least two numeric fields: discount_amount and final_amount, which must be mathematically correct according to the specified formulas.

Success Schema (Observational): The presence of additional fields in the success JSON is observational (acceptable unless they leak sensitive PII or internal system state).

Failure Oracle (Normative): When any condition C1–C5 fails, the API MUST NOT apply the coupon (no database state changes to usage limits, no discounted calculation returned).

Failure Schema & Status (Observational): The specific HTTP status code (e.g., 400, 401, 403, 422) and the exact JSON error schema returned on failure are purely observational.

Data Type Oracle (Observational): Because no strict integer-only rule is defined for total_amount, the API's acceptance of floats and its specific rounding behavior (floor, ceiling, nearest) for fractional discounts are observational properties, not normative failures.

B. Revised Decision Table (Authentication Gated)

Authentication (C4) is a strict gate. An unauthenticated caller must be rejected immediately to prevent probing for valid codes, thresholds, or expiration dates.

(T = True, F = False, - = Must Not Be Evaluated/Revealed)

Scenario C4 (Auth) C1 (Active) C2 (Not Exp) C3 (Total >= Min) C5 (Uses < Max) Normative Outcome

1. Unauthenticated F - - - - Reject. (Must not leak C1/C2/C3/C5 state)

2. Invalid/Inactive T F - - - Reject. Coupon invalid/not found

3. Expired Coupon T T F - - Reject. Coupon expired

4. Below Threshold T T T F - Reject. Minimum order amount not met

5. Max Uses Hit T T T T F Reject. Usage limit reached

6. Exact Threshold T T T Boundary T Success. Calculate & apply discount

7. Max - 1 Uses T T T T T (Max-1) Success. Calculate & apply discount

C. Trust-Boundary Table: JWT vs. Body user_id

The API requires a valid JWT for authentication (SEC-02, C4), but the request body accepts a user_id. The JWT identity is the authoritative source of truth.

JWT State Body user_id System Assessment Defensible (Normative) API Behaviors Critical Failure (Vulnerability)

Valid (User A) User A Match Proceed with C5 check for User A. N/A

Valid (User A) User B Mismatch (IDOR attempt)

Behavior 1: Reject request (e.g., 400/403).

Behavior 2: Ignore body ID, apply C5 for User A.

API evaluates C5 for User B (IDOR vulnerability).

Valid (User A) Missing Missing Field

Behavior 1: Ignore missing field, apply C5 for User A.

Behavior 2: Reject for strict schema failure.

N/A

Expired/Missing User A Unauthenticated Reject request immediately (Auth Failure). API bypasses JWT, trusts body ID.

D. Formula and Schema Invariants

When conditions C1–C5 evaluate to True, the following invariants MUST hold true in the system and response:

Schema Invariants:

Response.HasKey("discount_amount") == True

Response.HasKey("final_amount") == True

Type(discount_amount) == Numeric (Float or Integer)

Type(final_amount) == Numeric (Float or Integer)

Mathematical Invariants:

Core Rule: final_amount == total_amount - discount_amount

Fixed Logic: For fixed coupons (e.g., BIGBUY, VIP100), discount_amount == discount_value.

(Note: Because seeded fixed coupons have a min_order_amount greater than their discount_value, final_amount will always safely ev aluate to > 0).

Percent Logic: For percentage coupons (e.g., SAVE10), discount_amount == total_amount * (discount_value / 100).

Threshold Boundary Logic: If total_amount == min_order_amount, the mathematical invariants above must still execute identically w ithout off-by-one errors.

B■n ■ã nói

Generate the AI candidate test suite for POST /api/apply-coupon from the revised model. Produce exactly 45 distinct cases.

Use a compact Markdown table with:

AI_ID, Title, Preconditions, Request/header variation, Expected business outcome (APPLY/REJECT/OBSERVE), Automated oracle, Requir ement IDs/conditions, Technique, Priority.

Coverage rules:

- Individually cover code, total_amount, user_id, and Authorization: nominal, boundaries, missing, null, empty where relevant, an d wrong JSON types.

- Isolate C1-C5 and cover exact minimum, one below, one above; usage count max-1 and max; active/inactive, expired, nonexistent.

- Cover SAVE10 percent and BIGBUY/VIP100 fixed formula values, including documented thresholds and a fractional observational inp ut.

- Cover JWT: valid, missing, blank Bearer, malformed, invalid signature, expired, user A token with user B body ID, missing body user_id with valid JWT.

- Detect user_id usage-limit bypass: the safe outcome is reject mismatch or use JWT identity; using the supplied body identity is failure.

- Include replay/usage-record interactions and concurrent use-limit checks where relevant.

- Include SQL injection strings, oversized code, extra JSON properties, malformed JSON, duplicate JSON keys, and no leakage of st ack/database errors.

- Do not invent exact error HTTP codes or error fields.

- On success assert required numeric fields and exact formula invariants; additional fields are allowed unless sensitive.

- Mark concurrency, data seeding, time control, and out-of-band inspection requirements.

- All 45 are AI-generated candidates, not student-authored.

Afterward provide a coverage map and identify any cases that may be incomplete due to specification ambiguity.

Gemini ■ã nói

Here is the AI candidate test suite for POST /api/apply-coupon, generated strictly from the revised normative model.

AI Candidate Test Suite

AI_ID Title Preconditions Request/header variation Expected Outcome Automated Oracle Req/Cond Technique Priority

AI_01 Nominal Valid Request User 1 logged in, 0 uses of SAVE10
Valid JWT, body: {"code":"SAVE10", "total_amount":500000, "user_id":1} APPLY
Assert discount_amount: 50000, final_amount: 450000, Usage=1. C1-C5, Req-01 Base Nominal High

AI_02 Missing Auth Header User 1 Omit Authorization header REJECT No state change. Assert no DB error leak. C4, SEC-02 Boundary High

AI_03 Blank Bearer Token User 1 Authorization: Bearer (empty) REJECT No state change. C4, SEC-02 Boundary High

AI_04 Malformed JWT User 1 Authorization: Bearer <invalid chars> REJECT No state change. C4, SEC-02 Equivalence High

AI_05 Invalid Signature JWT User 1 JWT with tampered signature REJECT No state change. C4, SEC-02 Security High

AI_06 Expired JWT User 1 JWT with exp in the past REJECT No state change. C4, SEC-02 Boundary High

AI_07 Missing body user_id User 1 logged in Omit user_id from JSON body OBSERVE
OBSERVE if API applies via JWT identity or rejects for schema. Trust Bndry Equivalence Med

AI_08 IDOR: Body User B User 1 logged in Body user_id: 2 REJECT/APPLY System MUST NOT apply to User 2. Reject or apply to User 1. Trust Bndry Security High

AI_09 IDOR: Token maxed, Body 0 User 1 (max uses), User 2 (0 uses) JWT User 1, Body user_id: 2 REJECT
Assert coupon not applied. (Verifies JWT identity drives C5). C5, Trust Security High

AI_10 BIGBUY Nominal User 1, 0 uses BIGBUY {"code":"BIGBUY", "total_amount":600000, "user_id":1} APPLY
Assert discount_amount: 50000, final_amount: 550000. Formula Base Nominal High

AI_11 Non-existent Code User 1 code: "FAKE50" REJECT No state change. C1 Equivalence High

AI_12 Expired Code User 1 code: "EXPIRED" (expires 2020) REJECT No state change. C2 Boundary High

AI_13 Inactive Code Seed code INACTIVE10, active=0 code: "INACTIVE10" REJECT No state change. (Requires Seeding) C1 Equivalence High

AI_14 Code Case Sensitivity User 1 code: "save10" OBSERVE OBSERVE if normalized to SAVE10 or rejected. C1 Equivalence Low

AI_15 Code Empty String User 1 code: "" REJECT No state change. C1 Boundary Med

AI_16 Code Null User 1 code: null REJECT No state change. Schema Data Type Med

AI_17 Code Missing User 1 Omit code from body REJECT No state change. Schema Boundary Med

AI_18 Total Amount: Exact Min User 1, 0 uses SAVE10 total_amount: 300000 APPLY
Assert discount_amount: 30000, final_amount: 270000. C3. Formula Boundary High

AI_19 Total Amount: Min + 1 User 1, 0 uses SAVE10 total_amount: 300001 APPLY
Assert discount_amount: 30000.1 (or rounded), final: 269999.9. C3 Boundary High

AI_20 Total Amount: Min - 1 User 1 total_amount: 299999 REJECT No state change. C3 Boundary High

AI_21 Total Exact Min (BIGBUY) User 1, 0 uses BIGBUY total_amount: 500000 APPLY
Assert discount_amount: 50000, final_amount: 450000. C3. Formula Boundary High

AI_22 Total Exact Min (VIP100) User 1, 0 uses VIP100 total_amount: 300000 APPLY
Assert discount_amount: 100000, final_amount: 200000. C3. Formula Boundary High

AI_23 Total Amount: Float User 1 total_amount: 500000.55 OBSERVE OBSERVE rounding strategy or type rejection. Formula Data Type Med

AI_24 Total Amount: Zero User 1 total_amount: 0 REJECT No state change. C3 Boundary High

AI_25 Total Amount: Negative User 1 total_amount: -50000 REJECT No state change. C3 Equivalence High

AI_26 Total Missing User 1 Omit total_amount key REJECT No state change. Schema Boundary Med

AI_27 Total as String User 1 total_amount: "500000" OBSERVE OBSERVE type coercion or reject. Schema Data Type Low

AI_28 Total Null User 1 total_amount: null REJECT No state change. Schema Data Type Med

AI_29 Usage: 0 Prior (VIP100) User 1, 0 uses VIP100 VIP100 APPLY Applies, sets DB usage to 1. C5 Nominal High

AI_30 Usage: Max-1 (VIP100) User 1, 1 use VIP100 VIP100 APPLY Applies, sets DB usage to 2. (Requires Seeding) C5 Boundary High

AI_31 Usage: Max Hit (VIP100) User 1, 2 uses VIP100 VIP100 REJECT No state change. (Requires Seeding) C5 Boundary High

AI_32 Usage: Max Hit (SAVE10) User 1, 1 use SAVE10 SAVE10 REJECT No state change. (Requires Seeding) C5 Boundary High

AI_33 Concurrent Usage Limit User 1, 0 uses SAVE10 Fire 2 parallel threads with SAVE10 APPLY/REJECT
One MUST APPLY, one MUST REJECT. Final usage MUST be 1. (Concurrency) C5 Concurrency High

AI_34 SAVE10 Formula Nominal User 1 total_amount: 800000 APPLY Assert discount_amount: 80000, final_amount: 720000. Formula Equivalence High

AI_35 VIP100 Formula Nominal User 1 total_amount: 450000 APPLY Assert discount_amount: 100000, final_amount: 350000. Formula Equivalence High

AI_36 SQLi in Code User 1 code: SAVE10' OR '1'='1 REJECT No state change. NO SQL error leaked. SEC-05 Security High

AI_37 SQLi in User ID User 1 user_id: 1 OR 1=1 REJECT No state change. NO SQL error leaked. SEC-05 Security High

AI_38 Oversized Code Payload User 1 code: A x 5000 chars REJECT No state change. Robustness Boundary Med

AI_39 Extra JSON Properties User 1 {"code":"SAVE10", ..., "admin":true} OBSERVE OBSERVE if extra props are safely ignored. Schema Robustness Low

AI_40 Malformed JSON User 1 {"code":"SAVE10", (EOF) REJECT HTTP Error, no state change. Schema Robustness High

AI_41 Duplicate JSON Keys User 1 "code":"SAVE10", "code":"BIGBUY" OBSERVE OBSERVE which code system parses (or rejects). Schema Robustness Low

AI_42 User ID String User 1 user_id: "1" OBSERVE OBSERVE type coercion or rejection. Schema Data Type Low

AI_43 Code as Array User 1 code: ["SAVE10"] REJECT No state change. Schema Data Type Med

AI_44 Time Travel (Expiry C2) System time set: 2100-01-01 code: SAVE10 REJECT No state change. (Requires Time Control) C2 Boundary High

AI_45 Missing all params User 1 {} (empty JSON object) REJECT No state change. Schema Equivalence High

Coverage Map

Authentication (C4): AI_02 - AI_06.

Trust Boundary / IDOR: AI_07 - AI_09, AI_42.

Coupon Code (C1, C2): AI_11 - AI_17, AI_36 (SQLi), AI_38, AI_44 (Expiry Clock).

Total Amount (C3): AI_18 - AI_28.

Usage Limits (C5): AI_29 - AI_33 (Concurrency).

Formulas/Math: AI_01, AI_10, AI_18, AI_21, AI_22, AI_34, AI_35, AI_23 (float).

Robustness/Schema: AI_16, AI_17, AI_26-28, AI_38 - AI_41, AI_43, AI_45.

Ambiguities & Incomplete Cases (Observational Limits)

The following behaviors remain incomplete strictly from a specification standpoint and must be treated as observational during execution:

Error Status Codes: We assert REJECT, but the exact HTTP status codes (400, 401, 403, 404, 422) for failures cannot be predicted.

Fractional Math Handling: (AI_23) Standard currency rules usually dictate rounding to the nearest integer/cent. Because the spec provides no integer constraint, the rounding strategy is observational.

Strict Schema vs. Loose Parsing: (AI_14, AI_27, AI_39, AI_41, AI_42) Whether the API uses strict JSON type mapping (rejecting "1" for an int) or permissive coercion depends on the underlying web framework, which is currently undocumented.

IDOR Fallback Logic: (AI_07, AI_08) It is unknown whether the developer safely overrode the user_id with the JWT identity (APPLY) or threw a schema validation error due to the mismatch (REJECT). Both are valid security postures.

B■n ■ã nói

Targeted post-generation correction: several candidates assume POST /api/apply-coupon increments coupon usage (for example AI_01, AI_29, AI_30, AI_33). That state mutation is not specified. The endpoint description says it calculates the discounted total, while C5 only says the user's existing usage count must be below max. Consuming a coupon may occur only after successful checkout, and no such behavior is defined for this target endpoint.

Do not regenerate the whole suite. Produce a correction ledger for every affected test ID with:

Original ID, original problematic assertion, audit label (INVALID or INCOMPLETE), corrected precondition, corrected oracle, and rationale.

Rules:

- Pre-seed usage counts externally for max-1 and max conditions.

- Do not assert that apply-coupon increments usage unless an authoritative requirement is later provided.

- A repeat call at unchanged pre-seeded usage should produce the same eligibility result; treat this as idempotent calculation unless documentation says otherwise.

- Move concurrency around “consume the final use” to an integration test involving the actual usage-recording/checkout operation and mark it outside this endpoint's strict scope.

- Preserve C5 eligibility checks and JWT identity trust-boundary coverage.

Finally state the corrected total number of usable AI candidates after removing or replacing invalid duplicates.

Gemini ■ã nói

Correction Ledger

ID Original Problematic Assertion Audit Label Corrected Precondition Corrected Oracle Rationale

AI_01 Assert discount_amount: 50000, final_amount: 450000. Usage=1. INCOMPLETE
User 1 logged in, usage externally pre-seeded to 0 uses of SAVE10.
APPLY. Assert discount_amount: 50000, final_amount: 450000. Assert usage remains 0.
Endpoint calculates the discount but must be idempotent: it does not consume the coupon.

AI_29 Applies, sets DB usage to 1. INVALID
User 1 logged in, externally pre-seeded with 0 uses of VIP100. Execute API call twice consecutively.
APPLY. Both calls succeed returning identical discounts. Assert usage remains 0.
Validates idempotent eligibility evaluation for C5 without mutating state.

AI_30 Applies, sets DB usage to 2. *(Requires Seeding)* INVALID
User 1 logged in, externally pre-seeded with 1 prior use of VIP100 (Max - 1). Execute API call twice consecutively.
APPLY. Both calls succeed returning identical discounts. Assert usage remains 1. Proves that evaluating at the exact boundary (max-1) does not accidentally increment usage, which would wrongly block the second idempotent calculation.

AI_33 One MUST APPLY, one MUST REJECT. Final usage MUST be 1. *(Concurrency)* INVALID
(Moved to Integration) User 1 logged in, usage pre-seeded to 0. 2 parallel threads attempt the actual checkout/consume endpoint.
(Moved to Integration) One MUST SUCCEED, one MUST FAIL. Final usage = 1. Concurrency checks around "consuming the final use" belong in the checkout/transaction boundary, entirely outside the strict scope of this read-only calculation endpoint.

(Note: AI_31 and AI_32 correctly asserted "REJECT / No state change" for the Max Hit condition, meaning their original oracles remain valid under this read-only paradigm, provided their preconditions are externally seeded).

Final Usable AI Candidates

After correcting the overreaching state-mutation assumptions and migrating the concurrency race-condition test (AI_33) to the checkout integration suite, there are exactly 44 usable AI candidate tests remaining within the strict scope of the POST /api/apply-coupon requirements.

Pro

Gemini là AI và có th■ m■c sai sót.

Verbatim transcript — Import products

Trò chuv■n v■i Gemini

B■n ■ã nói

You are assisting with an individual university API-testing assignment. Work only from the authoritative requirements below. This is Phase 1: requirements analysis for exactly one target API. Do NOT generate test cases yet.

TARGET API

POST /api/admin/import-products

Authorization: Bearer <JWT>, admin role required.

JSON body accepted by the API specification:

{

"products": [

{

"name": "SP 1",

"price": 10000,

"description": "Mô t■ 1",

"imageUrl": "",

"category_id": 1

}

]

}

FR-16 BUSINESS REQUIREMENTS

Admin uploads a CSV whose extension is .csv.

Header must be: name,price,description,imageUrl,category_id.

Comma-containing fields are supported when double-quoted according to RFC 4180.

Before import, each row is validated: name must not be empty; price must be positive.

If any row has an error, the entire import must roll back (atomic all-or-nothing).

The system reports clearly how many rows succeeded, how many failed, and failure reasons.

RELATED PRODUCT RULES

FR-15 says product name is required and <=255 characters, price is required and >0, and category is required and chosen from an existing category.

SECURITY

FR-12 and SEC-02/SEC-03: every /api/admin/* API requires a valid JWT and role=admin. A normal user token must be rejected.

SEC-05: parameterized database queries.

The specification does not define exact HTTP status codes or exact success/error property names.

IMPORTANT SPECIFICATION MISMATCH

FR-16 describes CSV upload/parsing, but this target API receives an already parsed JSON products array. Distinguish API-level validation/atomicity tests from frontend CSV parser tests; do not attribute file-extension or RFC-4180 parsing directly to this JSON endpoint.

ASSIGNMENT EXPECTATIONS

Later tests must cover every nested parameter, arrays, cross-row interactions, state/atomicity, role escalation, injection, response schema, and robustness.

Produce:

1. Traceability split into API-level versus frontend/CSV-parser responsibilities.

2. Partitions and boundaries for products array and every product field.

3. Transaction/state model for validate -> commit or rollback.

4. Authorization decision table.

5. Response/count invariants without inventing property names.

6. Ambiguities requiring human review.

Do not inspect implementation behavior and do not generate individual cases yet.

Gemini ■ã nói

Here is the Phase 1 requirements analysis for the POST /api/admin/import-products endpoint. This analysis maps the business rules to the system boundaries, establishes the state models, and highlights ambiguities without generating executable test cases.

1. Traceability Split: API vs. Frontend Responsibilities

Because the business requirement (FR-16) describes a CSV upload but the target API accepts a pre-parsed JSON array, the testing responsibilities must be strictly bifurcated.

Feature / Requirement Frontend (CSV Parser) Responsibility API (Backend JSON) Responsibility

File Handling Validate .csv extension and file size limits. None (receives JSON payload).

CSV Formatting Enforce exact header (name,price,...); apply RFC 4180 parsing for double-quoted commas. None.

Payload Structure Convert parsed CSV rows into the JSON products array.
Reject malformed JSON; validate the products array schema.

Field Validation Surface UI errors before API call if possible (optional).
Enforce FR-15 and FR-16 constraints (name length, positive price, valid category).

Database State Display final status to the user.
Guarantee atomic all-or-nothing rollback; execute parameterized queries (SEC-05).

Authentication Pass the Bearer token in headers. Enforce JWT validity and admin role (FR-12, SEC-02/03).

2. Partitions and Boundaries

To validate the products array and its nested fields, the data must be partitioned into equivalence classes and boundary values.

Array-Level Partitions (products)

Empty state: Null array, empty array [].

Valid counts: 1 item (minimum), typical batch (e.g., 50 items).

Stress counts: Very large array (e.g., 10,000 items) to test timeout and payload limits.

Duplicate states: Array containing duplicate product names or identical payloads across multiple indexes.

Field-Level Partitions (per item in products)

Field Valid Partitions Invalid / Boundary Partitions

name Standard string, string with spaces, max length (255 chars).
Null, empty "" (FR-16), 256 chars (FR-15 boundary), SQL injection payloads (SEC-05).

price Positive integer/float (e.g., 1, 10000), large valid amount.
Null, negative number (-1), absolute zero (0), non-numeric string ("10k").

category_id Existing valid integer ID. Null, non-existent ID, negative ID, string/malformed type.

description Standard string, empty string (if optional). Null, excessively long string, HTML/script injection payloads.

imageUrl Valid URL, empty string "". Null, malformed URL, extremely long URL.

3. Transaction and State Model

FR-16 mandates an atomic, all-or-nothing import. The database state must never reflect a partial import.

Pre-Import State: Database holds baseline product count (X).

Validation Phase: The API iterates through the products array.

Condition A: All items pass schema, constraint, and referential (category) checks.

Condition B: One or more items fail validation.

Final State (A - Commit): The transaction commits. Database product count becomes X+N (where N is the number of items in the array).

Final State (B - Rollback): The transaction aborts. Database product count remains strictly X.

4. Authorization Decision Table

Based on FR-12, SEC-02, and SEC-03, the endpoint must be strictly protected.

Token State Decoded Role Action Expected Database State Change

Valid, active JWT admin Allow (process payload) Yes (if payload is valid)

Valid, active JWT user Reject (Forbidden) None

Expired JWT admin Reject (Unauthorized) None

Malformed/Tampered JWT Any / Unknown Reject (Unauthorized) None

No Token (Missing Auth) N/A Reject (Unauthorized) None

5. Response and Count Invariants

The API specification does not define specific property names for the response. However, we can establish mathematical invariants based on the requirement that the system reports successes, failures, and reasons.

Let T = Total number of products in the parsed JSON array.

Let S = Reported count of successful rows.

Let F = Reported count of failed rows.

Let D = Actual number of rows inserted into the database.

Invariants that must hold true for every response:

Total Count Check: T=S+F

Atomic Rollback Rule:

If F==0, then D=T (Complete Success).

If F>0, then D=0 (All-or-Nothing Rollback).

Error Tracing: The response must contain exactly F distinct error reasons mapped to the specific failing rows/indexes.

6. Ambiguities Requiring Human Review

Before proceeding to test case generation, the following ambiguities in the business requirements and API specification must be clarified:

"Success" Count during Rollback: If a batch of 10 items has 9 valid rows and 1 invalid row, the transaction rolls back (0 items saved). Should the API report S=9,F=1 (indicating rows that passed validation), or S=0,F=10 (indicating the final aborted state)?

Optionality of Fields: Are description and imageUrl strictly required, or can they be omitted/null? FR-15 explicitly requires name, price, and category, but is silent on the others.

Missing HTTP Status Codes: Because the spec lacks exact status codes, does a validation failure resulting in a rollback return a 400 Bad Request, 422 Unprocessable Entity, or a 200 OK with an error payload?

Duplicate Handling: How should the API handle a batch containing two products with the exact same name? Should it update the existing product, or throw a uniqueness constraint violation and roll back the whole batch?

B■n ■ã nói

Critically revise the import analysis before test generation.

Corrections:

1. Do not invent exact “Unauthorized/Forbidden” status categories. The normative result is reject before processing and no database change; record actual status/body.

2. “Exactly F distinct error reasons” is not supported: one row can have multiple validation errors, and the specification does not define response property names. Require at least one actionable reason for every rejected row, while recording the response shape for review.

3. Rollback creates two possible reporting interpretations: validation counts (e.g. 9 rows valid, 1 invalid, 0 committed) versus committed counts (0 succeeded). Do not force T=S+F until the semantic meaning of “success” is established. Require the unambiguous database invariant D=0 when any row is invalid.

4. SQL-injection-looking strings sent as a name/description are not automatically invalid inputs. SEC-05 requires treating them as literal parameter values without altering query structure or leaking database errors. A nonempty name payload can be accepted safely if all other rules pass.

5. Stored HTML/script in description concerns SEC-04 at display time. For this endpoint, assert no execution/response reflection and mark end-to-end escaping as a separate UI verification.

6. Separate normative field rules (name 1..255, price >0, existing category) from observational rules for description/imageUrl, batch-size limits, duplicate names, numeric floats, and extra properties.

7. Clarify atomicity verification must compare stable product identifiers/content as well as count to detect updates or partial writes hidden by a constant count.

Return:

A. Corrected API-level normative oracles.

B. Observational partitions.

C. Atomicity verification protocol.

D. Conservative response-reporting oracle.

E. Corrected authorization gate.

Still do not generate individual test cases.

Gemini ■ã nói

Here is the revised Phase 1 requirements analysis, incorporating the corrections to ensure strict alignment with the specification boundaries, security handling, and database invariants.

A. Corrected API-Level Normative Oracles

These are the strict, testable business rules (FR-15, FR-16) that dictate whether a product row is valid or invalid.

name: Must be provided, non-empty, and between 1 and 255 characters.

Security Note (SEC-05): Strings containing SQL injection payloads (e.g., ' OR 1=1; --) are normatively valid inputs. They must be treated as literal string values and accepted safely without altering database queries or leaking SQL errors.

price: Must be provided and strictly greater than 0.

category_id: Must be provided and correspond to an existing, valid category in the system.

Database Invariant (All Valid): If every row in the products array passes the normative rules, the transaction must commit all rows.

Database Invariant (Any Invalid): If one or more rows fail any normative rule, the transaction must roll back entirely (D=0 changes to the database).

B. Observational Partitions

Because the specification does not exhaustively define all system behaviors, the following partitions require observational testing to document the API's actual implementation and handle edge cases safely.

Unspecified Field Rules (description, imageUrl): Observe how the system handles missing, null, or empty string values, as well as extreme length limits.

Security Note (SEC-04): HTML/script injection payloads in description are observational for the API regarding reflection (they must not execute in the API response or crash the endpoint). End-to-end rendering safety is deferred to UI testing.

Batch Size Limits: Observe the maximum number of items (array length) and payload size the API can process before timing out or throwing a limit error.

Data Types and Formatting: Observe the handling of floating-point numbers for price (e.g., 100.50) versus integers, and stringified numbers.

Duplicate Handling: Observe API behavior when duplicate name values exist within the same array or conflict with existing database records (e.g., does it reject, update, or append?).

Schema Strictness: Observe whether unexpected extra properties in the JSON payload (e.g., "internal_code": "XYZ") are silently ignored or rejected.

C. Atomicity Verification Protocol

Verifying an all-or-nothing rollback requires more than checking the total row count, as a constant count might mask partial writes (e.g., an update overwriting an existing row).

Pre-State Snapshot: Record the total count of products in the database AND capture a snapshot of existing product identifiers and their core content (e.g., a hash or copy of the current state).

Execution: Submit a mixed payload (valid and invalid rows) to trigger a rollback.

Post-State Assertion:

The total database product count must exactly match the Pre-State count.

No new identifiers introduced by the payload can exist in the database.

No existing product content can be modified (verifying no partial updates occurred).

D. Conservative Response-Reporting Oracle

The API specification does not define exact HTTP status codes, property names, or the semantic meaning of "success" during a rollback. The oracle must evaluate the response dynamically:

Error Tracing: For every rejected row in the payload, the response must provide at least one actionable reason. (Note: A single row may have multiple errors, so error counts may exceed failed row counts).

Reporting Semantics: Record whether the response reports "success" based on validation counts (e.g., 9 valid, 1 invalid) or commit counts (0 committed, 10 failed). Do not force a specific mathematical formula (like T=S+F) until the API's reporting behavior is established.

Response Shape: Record the actual HTTP status code returned (whether 200, 400, 422, etc.) and the structure of the JSON response body for human review.

E. Corrected Authorization Gate

All requests to this endpoint must pass authentication (valid JWT) and authorization (admin role).

Valid JWT + Admin Role: The API must allow the request and proceed to payload processing (database changes depend on payload validity).

Missing, Invalid, Expired Token, or Non-Admin Role: The API must reject the request before processing.

Assertion: No database changes can occur.

Observation: Record the actual HTTP status code and response body returned by the API (do not invent or assume strict 401 Unauthorized or 403 Forbidden behaviors prior to testing).

B■n ■ã nói

Generate the AI candidate suite for POST /api/admin/import-products from the corrected model. Produce exactly 48 distinct cases.

Use a compact Markdown table with:

AI_ID, Title, Preconditions, Payload/auth variation, Expected business outcome (COMMIT/ROLLBACK/REJECT_BEFORE_PROCESSING/OBSERVE), Automated oracle, Requirement IDs, Technique, Priority.

Coverage requirements:

- Authorization: admin token; normal-user token; missing, blank, malformed, invalid-signature, and expired token. Assert no data change on all rejected requests.

- products container: missing, null, wrong type object/string, empty array, one row, several rows, large/stress batch observation.

- name: missing, null, empty, whitespace-only observation, lengths 1/255/256, Unicode, SQL-looking literal.

- price: missing, null, 0, negative, minimum positive, typical positive, float observation, numeric string observation, extreme numeric values.

- category_id: missing, null, existing, nonexistent, negative, wrong type; enforce FR-15 existing category.

- description and imageUrl: omitted/null/empty observations, long values, HTML/script string with separate UI escaping note, malformed URL observation.

- Cross-row atomicity: invalid row first/middle/last, several invalid rows, all valid, and snapshot/count/content verification.

- Response reporting: at least one actionable reason per rejected row, but do not invent status codes/property names or force success/failure count semantics.

- Security/schema: SQLi-like literal handling, extra properties, prototype-style properties, malformed JSON, duplicate JSON keys, large payload, concurrent duplicate imports observation, database error/stack leakage.

- Do not test .csv extension or RFC-4180 parsing as if handled by this JSON endpoint; list those only as out-of-scope companion UI/parser tests.

- Mark time/data seeding/concurrency/out-of-band DB requirements.

- All 48 are AI-generated candidates.

After the table provide coverage mapping and identify ambiguous/incomplete candidates.

Gemini ■ã nói

Here is the AI candidate suite containing exactly 48 test cases for the POST /api/admin/import-products JSON endpoint.

AI Candidate Test Suite

AI ID Title Preconditions Payload/Auth Variation Expected Outcome Automated Oracle Req IDs Technique Priority

TC-01 Valid Admin Token Admin JWT active, valid DB snapshot 1 valid product COMMIT DB +1 product; Response records success format FR-12, FR-16 EP High

TC-02 Normal User Token User JWT active 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged; Record response status/body FR-12, SEC-02 EP High

TC-03 Missing Token No JWT provided 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged; Record response status/body FR-12, SEC-02 EP High

TC-04 Blank Token Header Authorization: Bearer 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged FR-12, SEC-02 BVA Med

TC-05 Malformed Token Authorization: Bearer abc123invalid 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged FR-12, SEC-02 EG High

TC-06 Invalid JWT Signature JWT with tampered signature 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged SEC-02, SEC-03 Security High

TC-07 Expired Admin JWT Admin JWT past exp 1 valid product REJECT_BEFORE_PROCESSING DB snapshot unchanged SEC-02, SEC-03 BVA High

TC-08 Missing products key Admin JWT active Payload: {} REJECT_BEFORE_PROCESSING DB snapshot unchanged; Require actionable error FR-16 EP High

TC-09 products is null Admin JWT active {"products": null} REJECT_BEFORE_PROCESSING DB snapshot unchanged; Require actionable error FR-16 BVA Med

TC-10 products wrong type (obj) Admin JWT active {"products": {"name":"A"}} REJECT_BEFORE_PROCESSING DB snapshot unchanged; Require actionable error FR-16 EP Med

TC-11 products wrong type (str) Admin JWT active {"products": "[]"} REJECT_BEFORE_PROCESSING DB snapshot unchanged FR-16 EP Med

TC-12 products empty array Admin JWT active {"products": []} OBSERVE Record if rejected or accepted gracefully; DB unchanged FR-16 BVA Med

TC-13 Multiple valid rows DB has category 1 50 valid products COMMIT DB +50; Snapshot verifies all inserted FR-16 EP High

TC-14 Stress batch DB has category 1 5,000 valid products OBSERVE Record max payload size/timeout behavior FR-16 Stress Low

TC-15 name missing DB has category 1 Valid row but omit name key ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 EP High

TC-16 name null DB has category 1 name: null ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 BVA Med

TC-17 name empty string DB has category 1 name: "" ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 BVA High

TC-18 name whitespace only DB has category 1 name: " " OBSERVE Check if trimmed to empty and rejected FR-15 BVA Med

TC-19 name length 1 char DB has category 1 name: "A" COMMIT DB +1; Product saved exactly as "A" FR-15 BVA High

TC-20 name length 255 chars DB has category 1 name is 255 'A's COMMIT DB +1; Product saved correctly FR-15 BVA High

TC-21 name length 256 chars DB has category 1 name is 256 'A's ROLLBACK DB unchanged; ≥1 actionable error in response FR-15 BVA High

TC-22 price missing DB has category 1 Valid row but omit price key ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 EP High

TC-23 price null DB has category 1 price: null ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 BVA Med

TC-24 price exactly 0 DB has category 1 price: 0 ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 BVA High

TC-25 price negative DB has category 1 price: -100 ROLLBACK DB unchanged; ≥1 actionable error in response FR-15, FR-16 BVA High

TC-26 price min positive DB has category 1 price: 1 COMMIT DB +1; Price exactly 1 FR-15, FR-16 BVA High

TC-27 price float observation DB has category 1 price: 10.55 OBSERVE Check if DB supports float or throws/rounds FR-15 EP Med

TC-28 price numeric string DB has category 1 price: "100" OBSERVE Record if API auto-casts to int/float or rejects FR-15 EP Low

TC-29 price extreme max DB has category 1 price: 9999999999 OBSERVE Check for integer overflow or DB max limit FR-15 BVA Med

TC-30 category_id missing DB has category 1 Valid row, omit category_id ROLLBACK DB unchanged; ≥1 actionable error in response FR-15 EP High

TC-31 category_id null DB has category 1 category_id: null ROLLBACK DB unchanged; ≥1 actionable error in response FR-15 BVA Med

TC-32 category_id nonexistent DB has category 1 category_id: 999999 ROLLBACK DB unchanged; ≥1 actionable error in response FR-15 EP High

TC-33 category_id negative DB has category 1 category_id: -1 ROLLBACK DB unchanged; ≥1 actionable error in response FR-15 BVA Med

TC-34 category_id string type DB has category 1 category_id: "1" OBSERVE Record if API auto-casts string to int or rejects FR-15 EP Low

TC-35 description omitted DB has category 1 Omit description OBSERVE Commit expected; check if DB stores null or "" FR-16 EP Med

TC-36 imageUrl omitted DB has category 1 Omit imageUrl OBSERVE Commit expected; check if DB stores null or "" FR-16 EP Med

TC-37 description very long DB has category 1 description is 10k chars OBSERVE Check DB text limits/truncation behavior FR-16 Stress Low

TC-38 imageUrl malformed DB has category 1 imageUrl: "not-a-url" OBSERVE Check if strictly validated as URI format FR-16 EP Med

TC-39 HTML/script in desc DB has category 1 description: "<script>..." COMMIT DB +1; Assert stored literally (SEC-04 UI defer) SEC-04 Security High

TC-40 Atomicity: 1st row invalid DB has cat 1; rows: Bad, Good, Good 3 rows, price:0 on idx 0 ROLLBACK DB strictly D=0; content/snapshot unchanged FR-16 ST High

TC-41 Atomicity: Mid row invalid DB has cat 1; rows: Good, Bad, Good 3 rows, name:"" on idx 1 ROLLBACK DB strictly D=0; content/snapshot unchanged FR-16 ST High

TC-42 Atomicity: Last row invalid DB has cat 1; rows: Good, Good, Bad 3 rows, cat_id:-1 on idx 2 ROLLBACK DB strictly D=0; content/snapshot unchanged FR-16 ST High

TC-43 SQLi Literal in name DB has category 1 name: "' OR 1=1; --" COMMIT DB +1; literal string saved, queries parameterize SEC-05 Security High

TC-44 Extra properties in JSON DB has category 1 {"name":"A", "hack":true} OBSERVE Record if extra keys are stripped vs rejected Schema EP Low

TC-45 Malformed JSON payload Admin JWT active Missing closing bracket REJECT_BEFORE_PROCESSING Gateway/framework rejection; DB unchanged Schema EG Med

TC-46 Duplicate JSON keys DB has category 1 {"name":"A", "name":"B"} OBSERVE Check if JSON parser overrides vs errors out Schema EG Low

TC-47 Duplicate internal names DB has category 1 Array has 2 rows with same name OBSERVE Record if system updates, rejects, or appends DB EP Med

TC-48 Database error stack leak DB table locked/forced error 1 valid product OBSERVE API rejects; check response body lacks SQL trace Security Security High

Coverage Mapping

Authorization: Handled exhaustively in TC-01 through TC-07 (valid, missing, roles, token tampering, expiry).

Products Container: TC-08 through TC-14 map JSON structural boundaries and sizing limits.

Field Constraints (FR-15/FR-16): TC-15 to TC-34 methodically cover missing, null, type, boundary lengths, and normative validation checks (name limits, positive price, existing category).

Optional Fields: TC-35 to TC-39 test behaviors not explicitly defined in the spec, such as omitted fields, lengths, and format validation on images.

Atomicity (FR-16): TC-40 to TC-42 verify the "all-or-nothing" rule strictly with D=0 verifications using positional variants.

Security (SEC-02..05): Covered by TC-39 (stored scripts), TC-43 (SQL injection syntax as literal input), and TC-48 (error leakage prevention).

Ambiguous / Incomplete Candidates (OBSERVE)

The tests marked as OBSERVE require the QA engineer to run the payload and record how the implementation behaves, as the requirements are currently silent:

Empty / Missing state defaults: How are missing optional fields (TC-35, TC-36) or whitespace-only names (TC-18) treated?

Type Casting: Does the API accept "100" as a price and cast it, or reject it for being a string (TC-28, TC-34)? Does it support floats or strictly integers (TC-27)?

Payload Strictness: Will extra JSON keys (TC-44) or duplicated keys (TC-46) be rejected (strict schema) or ignored?

Duplicate Business Logic: TC-47 tests if identical names result in a rollback, a duplicate entry, or an upsert (update).

Reporting Semantics: The "Automated Oracle" deliberately observes the response shapes (counts, error properties) across all ROLLBACK instances rather than asserting a rigid {"success": X, "fail": Y} format.

Out of Scope (UI/Frontend Responsibilities)

CSV File Parsing: Validating .csv file extensions, MIME types, and file size limits.

RFC-4180 Parsing: Validating double-quoted commas (e.g., "Mô t■, có d■u ph■y") converting properly to a string. This is tested at the frontend CSV parser layer before the JSON API request is ever generated.

B■n ■ã nói

Post-generation audit correction. Do not regenerate the 48-row table.

First produce a correction ledger for affected original IDs:

- TC-08 through TC-12 authenticate successfully; invalid container/schema is rejected during payload validation, not “before processing.” Correct the outcome terminology and oracle.

- TC-27 price 10.55 is a positive JSON number. FR-15 says price is a number >0 and does not require an integer. Explain whether this should be VALID/COMMIT normatively, while rounding/display remains observational.

- TC-39 must not claim safe UI rendering merely because the API stores a literal. Keep a companion end-to-end UI escaping check.

- TC-43 black-box insertion of an SQL-looking literal supports behavior evidence but does not prove all queries are parameterized; source/database review is required for SEC-05.

- TC-48 requires controlled white-box fault injection and cannot be executed as an ordinary black-box case.

For each affected ID return Original ID, audit label VALID/INVALID/INCOMPLETE, corrected expected outcome/oracle, and rationale.

Then add exactly 8 supplemental AI-generated candidates, IDs TC-49 through TC-56, covering the omissions:

1. Unicode Vietnamese product name.

2. description = null and imageUrl = null in a focused case.

3. Multiple invalid rows requiring reasons for each rejected row.

4. All-valid three-row atomic commit with snapshot verification.

5. Prototype-style __proto__/constructor extra properties.

6. Concurrent identical import requests as an observational idempotency/duplication case.

7. category_id boolean/array wrong JSON types.

8. Normal-user token plus SQL-looking valid product to prove authorization gates validation and makes no DB change.

End with the revised usable candidate count and updated coverage note.

Gemini ■ã nói

Correction Ledger for Affected Original IDs

Original ID Audit Label Corrected Expected Outcome / Oracle Rationale

TC-08 to TC-12 INVALID (Terminology)

REJECT_AT_VALIDATION

Oracle: DB unchanged; response returns validation error.

The requests successfully pass the authorization gate (valid admin JWT). The rejection occurs during JSON schema/payload parsing, not "before processing."

TC-27 INVALID (Outcome)

COMMIT

Oracle: DB +1; price recorded. Observation remains for rounding/display.

FR-15 strictly requires price >0. A positive float (10.55) satisfies this business rule normatively and must be accepted. How the DB maps/rounds the float is observational.

TC-39 INCOMPLETE (Oracle claim)

COMMIT

Oracle: DB +1; literal string saved. (Flag for separate E2E UI testing).

Storing the script literal in the DB proves API acceptance, but it cannot definitively prove safe UI rendering (SEC-04). End-to-end UI escaping must be verified in a separate layer.

TC-43 INCOMPLETE (Verification method)

COMMIT

Oracle: DB +1; verify literal string saved. Require source/DB review.

A black-box API test can verify the payload is accepted without crashing, but it cannot conclusively prove parameterization (SEC-05). It must be paired with white-box code review or DB query log inspection.

TC-48 INVALID (Execution method)

OBSERVE (White-box execution)

Oracle: Induce DB fault; assert API response lacks stack trace.

A standard black-box valid payload cannot reliably trigger a database error. This requires controlled fault injection (e.g., locking the table or mocking the DB connection) to verify SEC-04/SEC-05 error leakage prevention.

Supplemental AI Candidates (TC-49 to TC-56)

AI ID Title Preconditions Payload/Auth Variation Expected Outcome Automated Oracle Req IDs Technique Priority

TC-49 Unicode Vietnamese name DB has category 1, Admin JWT name: "S■n Ph■m Có D■u" COMMIT
DB +1; snapshot verifies exact Unicode text preserved. FR-15 EP High

TC-50 description & imageUrl explicitly null DB has category 1, Admin JWT Both fields sent as null in a valid row OBSERVE
Check if DB accepts nulls for optional fields vs requiring empty strings. FR-16 BVA Med

TC-51 Multiple invalid rows & reasons DB has category 1, Admin JWT
3 rows, all invalid (e.g., empty name, negative price, bad cat) ROLLBACK
DB D=0; Response contains at least one reason for each rejected row. FR-16 ST High

TC-52 All-valid multi-row snapshot DB has category 1, Admin JWT 3 completely valid rows COMMIT
DB D=3; Content snapshot perfectly matches all 3 parsed payloads. FR-16 ST High

TC-53 Prototype pollution payload DB has category 1, Admin JWT Add __proto__: {"admin": true} to JSON row ROLLBACK
DB D=0 or ignores property; no prototype override occurs. Schema Security High

TC-54 Concurrent identical imports DB has category 1, Admin JWT Send exact same valid batch concurrently on 2 threads OBSERVE
Record race conditions, duplicates, or constraint violations. DB Stress Med

TC-55 category_id wrong JSON types DB has category 1, Admin JWT category_id: true or [1] ROLLBACK
DB D=0; Schema validates types before DB execution. FR-15 EP High

TC-56 Auth gate vs SQLi parsing User JWT (non-admin) active Valid product, but name contains SQLi literal
REJECT BEFORE PROCESSING DB D=0; Proves auth rejection occurs before payload parsing/SQL risk. FR-12, SEC-02 ST High

Revised Usable Candidate Count and Coverage Note

Revised Usable Candidate Count: 56 distinct candidates. (The 48 original candidates remain active with the 5 ledger corrections applied to their execution/oracles, plus the 8 new supplemental candidates).

Updated Coverage Note:

The suite now explicitly covers Vietnamese localization constraints (TC-49), concurrent idempotency states (TC-54), prototype pollution schema vulnerabilities (TC-53), and explicit multi-row failure reason reporting (TC-51). Furthermore, the boundary between authentication and validation is proven (TC-56), and the distinction between black-box observable API behavior and white-box security verification (parameterization logs, fault injection) is formally documented in the oracles.

Pro

Gemini là AI và có th■ m■c sai sót.